

Low-Level Design (LLD)

Credit Risk Prediction Platform

1 Introduction

This document provides the low-level design of the Credit Risk Prediction Platform. It includes API specifications, data flow, module-level design, and implementation details.

2 System Components

2.1 Frontend (Streamlit)

- Captures user input
- Sends HTTP requests to backend API
- Displays prediction results

2.2 Backend API (FastAPI)

- Handles inference requests
- Validates input data
- Returns prediction output

2.3 Model Layer

- Loads trained model from `artifacts/model.pkl`
- Performs prediction using `predict_proba()`

2.4 Data Pipeline

- Feature engineering
- Train-test split
- Model training and evaluation

3 API Specification

3.1 POST /predict

Description: Returns probability of loan default and decision.

Request Format:

```
{
  "data": {
    "feature_1": value,
    "feature_2": value,
    ...
  }
}
```

Response Format:

```
{
  "default_probability": float,
  "decision": "APPROVE" | "REJECT"
}
```

Processing Steps:

1. Receive JSON input
2. Convert to Pandas DataFrame
3. Align input features with model schema
4. Fill missing features with default values
5. Call model for prediction
6. Return probability and decision

3.2 GET /metrics

Description: Exposes Prometheus metrics.

Response: Plain text metrics format.

4 Data Flow

1. User enters input in UI
2. UI sends request to FastAPI
3. API processes request and loads model
4. Model predicts probability
5. API returns response
6. Prometheus scrapes metrics

5 Module Design

5.1 data.py

- Loads raw data
- Performs preprocessing
- Splits into train/test sets

5.2 features.py

- Feature engineering
- Aggregations from external datasets

5.3 model.py

- Model training using LightGBM
- Cross-validation
- Feature importance generation

5.4 evaluate.py

- Computes business loss
- Finds optimal threshold
- Generates evaluation metrics

5.5 train.py

- Orchestrates full training pipeline
- Logs experiments to MLflow

6 Error Handling

- Invalid input format handled by FastAPI validation
- Missing features defaulted to zero
- API failures handled at UI level

7 Logging

- Pipeline logs using Python logging module
- Tracks execution stages and errors

8 Configuration

- All parameters stored in `config.yaml`
- Includes model, training, threshold, and business configs

9 Pipeline Design (DVC)

- Stage 1: Feature Engineering
- Stage 2: Train-Test Split
- Stage 3: Model Training
- Stage 4: Evaluation

10 Conclusion

The system is designed with clear separation of concerns. Each module has a well-defined responsibility, ensuring maintainability and scalability.